

Lab6 实验报告

Lab6 实验报告：中断与异常支持

1. 实验要求对照

Lab6 要求 CPU 支持 RISC-V 机器态 trap 相关机制。根据 Wiki，本次实验必须先理解特权架构，并完成以下内容：

- 实现异常：指令地址不对齐、数据地址不对齐、非法指令、ECALL
- 实现中断：时钟中断、外部中断、软件中断
- 实现 `mret`
- 发生 trap 时正确更新 `mepc`、`mcause`、`mstatus` 和当前 mode
- trap 后跳转到 `mtvec`
- trap/mret 后清除流水线，避免旧指令或旧访存响应污染新控制流
- 运行 `make test-lab6`
- 本次 Lab 不要求上板

Wiki 中还提到 MMU 缺页异常属于 bonus。本实现已在 Lab5 Sv39 MMU 基础上接入页错误 trap，因此公开要求和 bonus 方向都覆盖到了。

2. Trap 总体设计

本实现把中断和异常统一成 trap 流程处理。同步异常来自正在提交的指令，异步中断来自 `trint`、`exint`、`swint` 三根输入线。trap 发生后，CSR 模块完成架构状态更新，前端和 MMU 负责丢弃旧控制流下的状态。

一次 trap 的核心动作如下：

```
mepc          <- trap 对应 PC
mcause[63]    <- 0 表示异常，1 表示中断
mcause[62:0]  <- trap 类型
mstatus.MPIE  <- mstatus.MIE
mstatus.MIE   <- 0
mstatus.MPP   <- trap 前 mode
mode          <- M-mode
next_pc       <- mtvec
```

对应的返回由 `mret` 完成：

```
mstatus.MIE  <- mstatus.MPIE
mstatus.MPIE <- 1
mode        <- mstatus.MPP
```

```
mstatus.MPP <- 0
next_pc      <- mepc
```

代码上，trap 的核心状态更新集中在 `vsrc/src/core/core_csr.sv`，trap 信号从 `vsrc/src/core/core_commit.sv` 接入，前端 flush 和 difftest 接口在 `vsrc/src/core.sv` 中完成。

3. 异常实现

3.1 ECALL

ECALL 的 `mcause` 由当前特权级决定：

来源特权级 `mcause`

```
U-mode    8
S-mode    9
M-mode   11
```

CSR 模块中使用 `get_ecall_cause()` 生成对应 `cause`。ECALL trap 时：

```
mepc      <- wb_r.pc
mcause    <- get_ecall_cause(mode)
mtval     <- 0
mode      <- M-mode
pc        <- mtvec
```

3.2 非法指令

非法指令 trap 时写入：

```
mepc      <- wb_r.pc
mcause    <- 2
mtval     <- 原始非法指令编码
```

这样 handler 可以通过 `mtval` 判断触发异常的指令内容。

3.3 地址不对齐

本实现覆盖三类地址不对齐：

类型	mcause	mtval
指令地址不对齐	0	跳转目标地址
load 地址不对齐	4	load 地址
store 地址不对齐	6	store 地址

执行级计算跳转目标后检测 `ex_next_pc[1:0]`。若目标不对齐，生成 `ex_instr_misalign`，同时禁止错误跳转继续刷新前端。数据地址不对齐使用访存地址检测，并在 `commit/CSR` 路径中生成对应 trap。

3.4 页错误

MMU 页表遍历发现无效 PTE 或非叶子错误时产生 page fault，并通过 `fault_vaddr` 和 `fault_is_insn` 送入 core：

类型	mcause	mtval
instruction page fault	12	fault_vaddr
load page fault	13	fault_vaddr
store page fault	15	fault_vaddr

这部分属于 Wiki 中的 bonus，但对 Lab5/Lab6 的特权级和分页组合测试也有实际作用。

3.5 优先级处理

同步异常已经到达 WB/commit 时，不能被同一拍的异步中断覆盖。因此 CSR 中加入

`sync_trap_or_mret`：

```
assign sync_trap_or_mret = wb_ecall || wb_illegal || wb_misalign_instr ||
                             wb_misalign_data || mmu_trap || wb_mret;
```

中断只有在没有同步 trap/mret 抢占时才进入 trap。这使异常优先级更符合“当前提交指令先产生确定架构行为”的原则。

4. 中断实现

4.1 中断 pending 与 enable

三类中断信号映射到 `mip`：

输入	mip 位	mie 位	mcause
swint	3	3	0x8000000000000003
trint	7	7	0x8000000000000007
exint	11	11	0x800000000000000b

中断进入 M-mode 的条件按 Wiki 实现：

(当前 mode 不是 M-mode，或 `mstatus.MIE = 1`)
 并且
`mip[i] = 1` 且 `mie[i] = 1`

实现中没有检测中断信号边沿，而是把外部中断输入组合到 `mip` 对应位，这符合 Wiki 中“不要检测 `posedge/negedge`”的要求。

4.2 CSR 写回后的即时判定

Wiki 指出中断条件依赖 mip、mie、mstatus，并且这些 CSR 被写入后需要重新 evaluate。当前实现对 mstatus 和 mie 使用本拍有效值：

```
assign intr_mstatus =
    (wb_r.valid && wb_r.csr_wen && (wb_r.csr_addr == CSR_MSTATUS))
    ? wb_r.csr_wdata
    : csr_mstatus;

assign intr_mie =
    (wb_r.valid && wb_r.csr_wen && (wb_r.csr_addr == CSR_MIE))
    ? wb_r.csr_wdata
    : csr_mie;
```

这样 `csrsi mstatus, 8` 打开 MIE 后能及时响应中断，而 `csrci mstatus, 8` 清除 MIE 后不会因为旧值重复进中断。

4.3 精确中断 PC

中断是异步事件，前端 PC 可能已经取到后续指令。若直接把 `fetch_pc` 写入 `mepc`，中断返回后可能跳过尚未提交的指令。

因此 `mepc` 选择流水线中最老的未提交指令 PC：

```
assign intr_fetch_pc = mem_r.valid ? mem_r.pc :
                        ex_r.valid ? ex_r.pc :
                        id_r.valid ? id_r.pc :
                        fetch_pc;
```

这个设计保证中断前已经提交的指令不会重复执行，尚未提交的指令不会丢失。公开测试中的 `m_trap` 对这个点很敏感。

5. mret 与特权级恢复

`mret` 在 CSR 模块中恢复中断使能和特权级：

```
MIE  <- MPIE
MPIE <- 1
MPP  <- 0
mode <- 原 MPP
PC   <- mepc
```

`mret_redirect` 会通知前端跳转到 `mepc`，同时清除流水线中旧路径的指令，避免 trap handler 后续指令继续进入执行级。

6. 流水线与 MMU flush

trap、mret、执行级跳转都会改变控制流。实现中在这些事件发生时清理前端状态：

- `fetch_pending`

- `fetch_req_pc`
- `fetch_redirect_pending`
- `fetch_buf_valid`

同时加入 `fetch_redirect_bubble`，屏蔽 `redirect` 后短时间内返回的旧取指响应。

MMU 也接入 `flush_mmu_o`:

```
assign flush_mmu_o = trap_redirect || mret_redirect || ex_flush_front;
```

MMU 收到 `flush` 后清理 `page walk` 状态、保存的虚拟地址、PTE、`fault` 地址、`fault` 类型和 `trap_pending`。这可以避免旧 U-mode 请求或旧 `page fault` 在切到 M-mode `trap handler` 后污染新状态。

7. Difftest 状态

Lab6 官方测试暂时不使用 `Difftest`，但 Lab5 和后续测试需要 CSR 状态一致。本实现将 `DifftestCSRState.privilegeMode` 接到真实 `privilege mode`:

```
.privilegeMode(privilege_mode_diff)
```

并让 `privilege_mode_diff` 使用 `next_privilege_mode`，避免 `trap/mret` 提交点上报滞后一拍的特权级。

8. 测试结果

8.1 Lab6

在 WSL 中运行:

```
make test-lab6
```

关键输出如下:

```
Single test passed.
Run sys-test
trap here, epc 8000600c, cause 8
Test ecall_u [OK]
trap here, epc 80006028, cause 0
Test instr_misalign [OK]
trap here, epc 80006040, cause 4
Test load_misalign [OK]
trap here, epc 80006050, cause 6
Test store_misalign [OK]
trap here, epc 800060a8, cause 8000000000000007
Test timer_intr [OK]
trap here, epc 80006090, cause 8000000000000003
Test software_intr [OK]
Test m_trap [OK]
Privileged test finished.
```

Wiki 中说明后续循环输出 `m_trap_test [X] / ---TEST FAILED---` 属于正常现象，可以手动退出。当前实现已经能输出 `Test m_trap [OK]` 和 `Privileged test finished.`，因此公开 Lab6 测试通过。

8.2 前五个 Lab 回归

为确认没有破坏前五个实验，回归结果如下：

```
Lab1: HIT GOOD TRAP at pc = 0x80010004
Lab2: HIT GOOD TRAP at pc = 0x8001ffff
Lab3: HIT GOOD TRAP at pc = 0x80000030
Lab4: HIT GOOD TRAP at pc = 0x8001ffff
Lab5: Return from init! Test passed
```

Lab5 和 Lab6 在打印通过信息后不会立即自然退出，因此验证时使用 `timeout` 截断仿真进程。判断通过与否以输出中的通过信息为准。

9. 提交说明

本次 Lab6 不要求上板。提交包通过：

```
make handin
```

生成，报告同时提供 `docs/report.md` 和 `docs/report.pdf`，并保留 `docs/lab6_report.md` 作为同内容的 Lab6 专项报告。

10. 小结

本实现覆盖了 Wiki 中列出的异常、中断和 `mret` 行为，并额外接入了 MMU 缺页异常。关键设计点包括：

- WB/commit 收口同步异常
- CSR 统一维护 trap 架构状态
- 中断按 `mip`、`mie`、`mstatus.MIE` 判定
- CSR 写回后的 `mstatus/mie` 参与同拍中断判定
- 中断 `mepc` 保存流水线中最老的未提交指令 PC
- trap/mret 后清理前端和 MMU 旧状态
- 同步异常和 `mret` 不被同拍异步中断覆盖

公开测试 `make test-lab6` 已通过，且 Lab1-Lab5 回归保持通过。